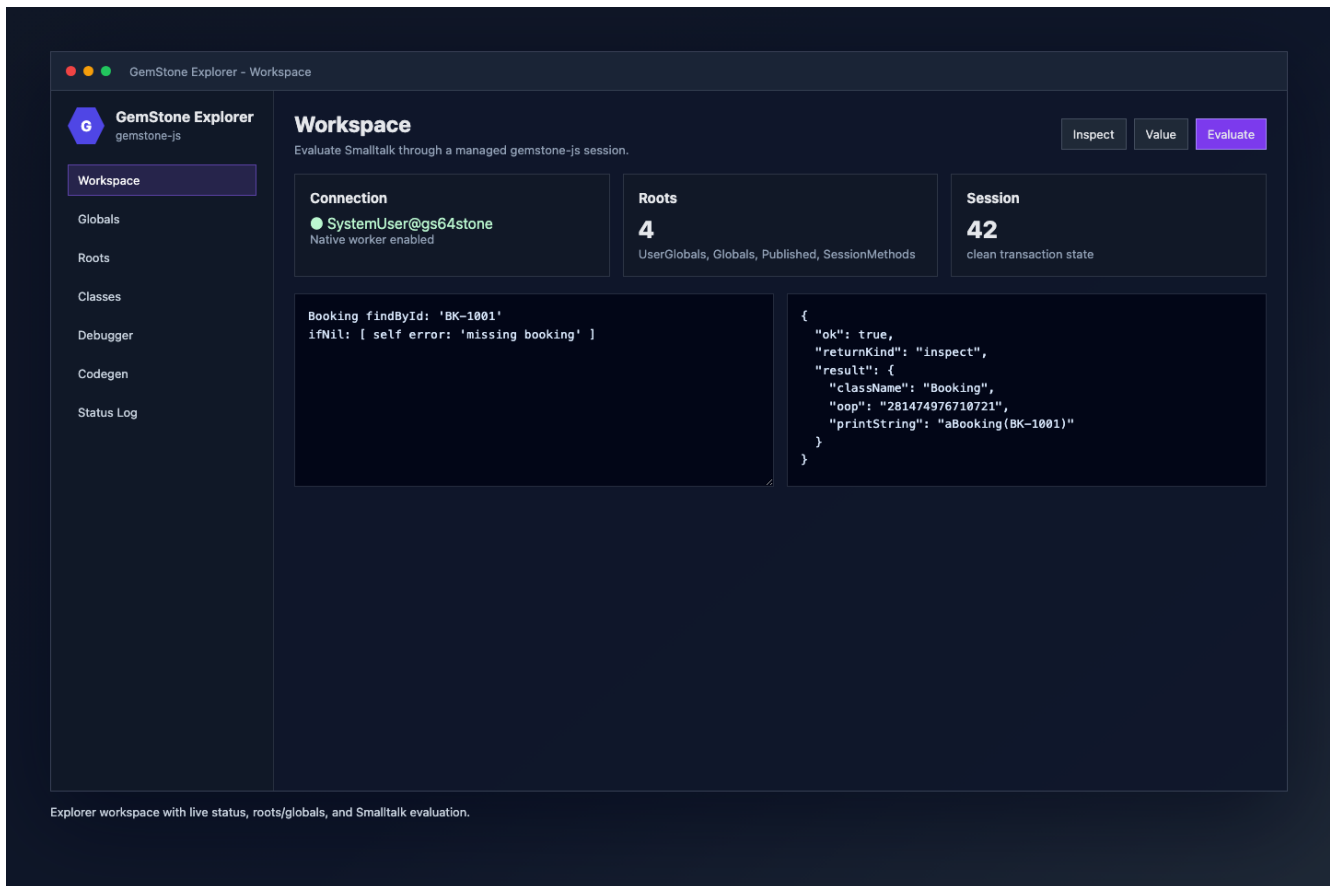


Medium Article: gemstone-js Workbench

Building a JavaScript Workbench for GemStone/S

A practical look at `gemstone-js`, the Explorer UI, and the VS Code extension that turns GemStone/S into a JavaScript-native development workflow.



GemStone/S has always rewarded developers who are comfortable living close to the object database. The hard part for many modern teams is not the database model itself. It is the tooling boundary around it: JavaScript services, TypeScript wrappers, npm release flows, CI checks, and editor workflows all need a clear path into the Stone.

`gemstone-js` is that bridge. It is an async TypeScript client for GemStone/S, with a native GCI backend, high-level session helpers, generated wrappers, web adapters, query helpers, migration tooling, release checks, and a browser-based Explorer. The newer VS Code workbench wraps that Explorer so developers can inspect, evaluate, debug, and browse GemStone objects without leaving their editor.

This article walks through the current shape of the project and the decisions behind the workbench.

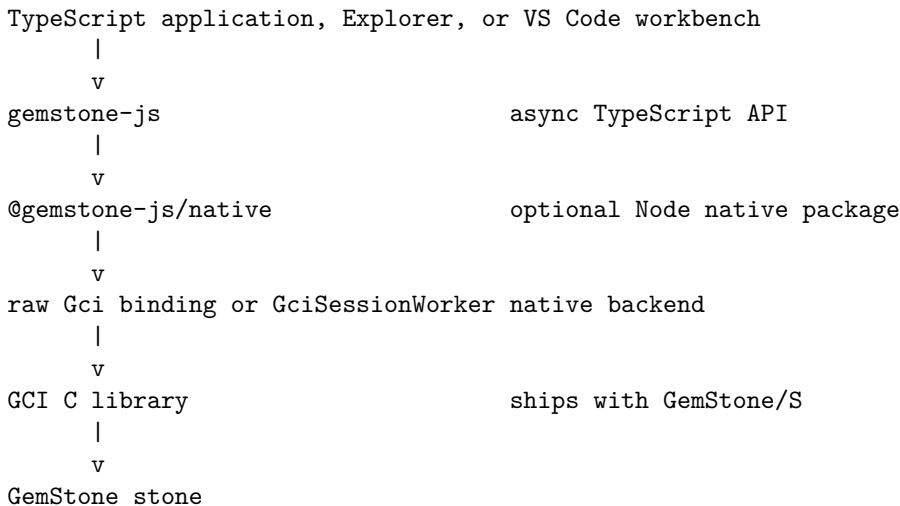
What GemStone/S Gives JavaScript Teams

GemStone/S 64 is an object database and application server for Smalltalk systems. It stores live objects, lets many sessions work concurrently, and lets clients send Smalltalk messages directly to objects in the Stone. The important difference from a relational database is that the primary unit is object identity, not a row.

That matters for JavaScript because modern services often want a typed, package-managed, CI-friendly way to reach those live objects. `gemstone-js` is not trying to turn GemStone into a SQL-shaped store. It gives TypeScript code a clear way to connect, send selectors, retain remote object handles, build dictionaries and arrays, inspect objects, and keep those interactions testable.

The Architecture

The shape mirrors the same split used by `gemstone-rs`: isolate low-level GCI work, keep the application API higher level, and make tools use the same tested core.



The layers have different jobs:

- `gemstone-js` provides `Session`, `TypedOop<T>`, persistent roots, dictionaries, arrays, queries, migrations, web adapters, codegen, and release checks.
- `@gemstone-js/native` loads and calls the GemStone GCI library from Node, including the session-worker backend.
- `GciSessionWorker` queues calls for one session through a dedicated native worker boundary.
- The Explorer provides local browser UI for doctor checks, evaluation, inspection, class browsing, debugging, and codegen preview.
- The VS Code workbench adds editor commands and tree views that wrap the Explorer and the same session API.

Keeping those boundaries separate makes the project easier to ship. Mock-runtime tests can run without a Stone, installed examples can be verified from an npm package, and live tests can be enabled only when `GS_RUN_LIVE=1` is set.

Install and First Check

For local TypeScript development:

```
npm install gemstone-js
```

For real GemStone/S access from Node, install the optional native package as well:

```
npm install gemstone-js @gemstone-js/native
```

The native package is optional so docs builds, mock-runtime tests, and browser-adjacent tooling can install `gemstone-js` without a GemStone client library. A production Node process that talks to a Stone should pin both package versions together.

The standard environment is:

```
export GS_STONE=gs64stone
export GS_NETLDI=netldi
export GS_HOST=localhost
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

Existing GemStone bridge shells can keep using aliases such as `GS_USER`, `GS_PASS`, `GS_NETLDI_HOST`, `GS_NETLDI_NAME_OR_PORT`, and `GS_SERVICE`. The canonical JavaScript names win if both are present.

Run the setup check before writing code:

```
gemstone-js-doctor
gemstone-js-doctor --live
gemstone-js-doctor --live --json
```

The non-live doctor checks runtime configuration, credentials presence, native package importability, and worker-backend shape. The live doctor logs in and evaluates `1 + 1`. JSON output masks secrets, which makes it usable in diagnostics and CI logs.

Installed examples are discoverable too:

```
gemstone-js-examples --commands
gemstone-js-examples --plan first-session
gemstone-js-examples --plan typed-codegen
```

First Login

The smallest live program is intentionally plain TypeScript:

```
import { Session } from "gemstone-js";

await using session = await Session.connect(Session.configFromEnv());

const value = await session.eval("3 + 4");
console.log(value);
// 7
```

`Session.configFromEnv()` reads the same `GS_*` variables used by the doctor, examples, Explorer, and VS Code workbench. The `await using` form logs out at the end of the block in runtimes that support explicit resource management; calling `await session.logout()` directly is also fine.

OOPs, Values, and Handles

GemStone objects are identified by OOPs. `gemstone-js` keeps that visible instead of hiding it behind a generic ORM.

```
import { Session, type TypedOop } from "gemstone-js";

interface Booking {
  id: string;
  status: string;
}

await using session = await Session.connect(Session.configFromEnv());

const booking: TypedOop<Booking> = await session
  .classRef<Booking>("Booking")
  .sendObject("find:", "B-1001");

const printString = await booking.printString();
```

```
const status = await booking.sendValue<string>("status");
await booking.release();
```

The mapping is explicit:

- `nil` comes back as `null`.
- `true` and `false` come back as JavaScript booleans.
- `SmallInteger` comes back as `bigint`.
- `Character`, fetched `String`, and fetched `Symbol` values come back as strings.
- `Array` readback uses bounded JavaScript arrays through `arrayOopToValues()`.
- `Dictionary` readback uses `GsDict` handles or plain snapshots through dictionary helpers.
- Other live objects come back as raw `Oop` values or retained `TypedOop<T>` handles.

That gives callers a choice. Use `performValueWith()` when a marshalled JavaScript value is enough, `performWith()` when raw identity matters, and `performObjectWith()` or `classRef().sendObject()` when the result should be retained as a typed handle.

Transactions and Request Lifetimes

GemStone sessions are transactional. At the lowest level, the API gives direct control:

```
await session.commit();
await session.abort();
const dirty = await session.needsCommit();
const active = await session.inTransaction();
```

For application work that should retry on commit conflicts, use the transaction helper:

```
import { PersistentRoot, runTransactionWithRetry, Session } from "gemstone-js";

await runTransactionWithRetry(async (session) => {
  const root = PersistentRoot.userGlobals(session);
  await root.setValue("GemStoneJsArticleCounter", Date.now());
}, {
  config: Session.configFromEnv(),
  attempts: 3,
});
```

For web services, request scopes and adapters make the session lifetime explicit. Express, Fastify, Hono, and Fetch adapters can commit, abort, or leave transaction control manual according to the route's policy. That keeps the GemStone transaction boundary aligned with the JavaScript request boundary.

The Goal

The goal is not to hide GemStone behind a generic ORM. GemStone is an object database, and the API should preserve that reality.

The JavaScript side needs to make common work explicit:

- connect to a Stone with clear environment variables
- send selectors and evaluate Smalltalk
- marshal JavaScript strings, numbers, arrays, dictionaries, and typed object handles
- inspect OOPs and object structure
- work with persistent roots, globals, dictionaries, arrays, and ordered collections
- generate typed wrappers from manifests or decorators
- test locally with a mock runtime and opt into live Stone regression tests
- run production calls through a safer native session-worker backend

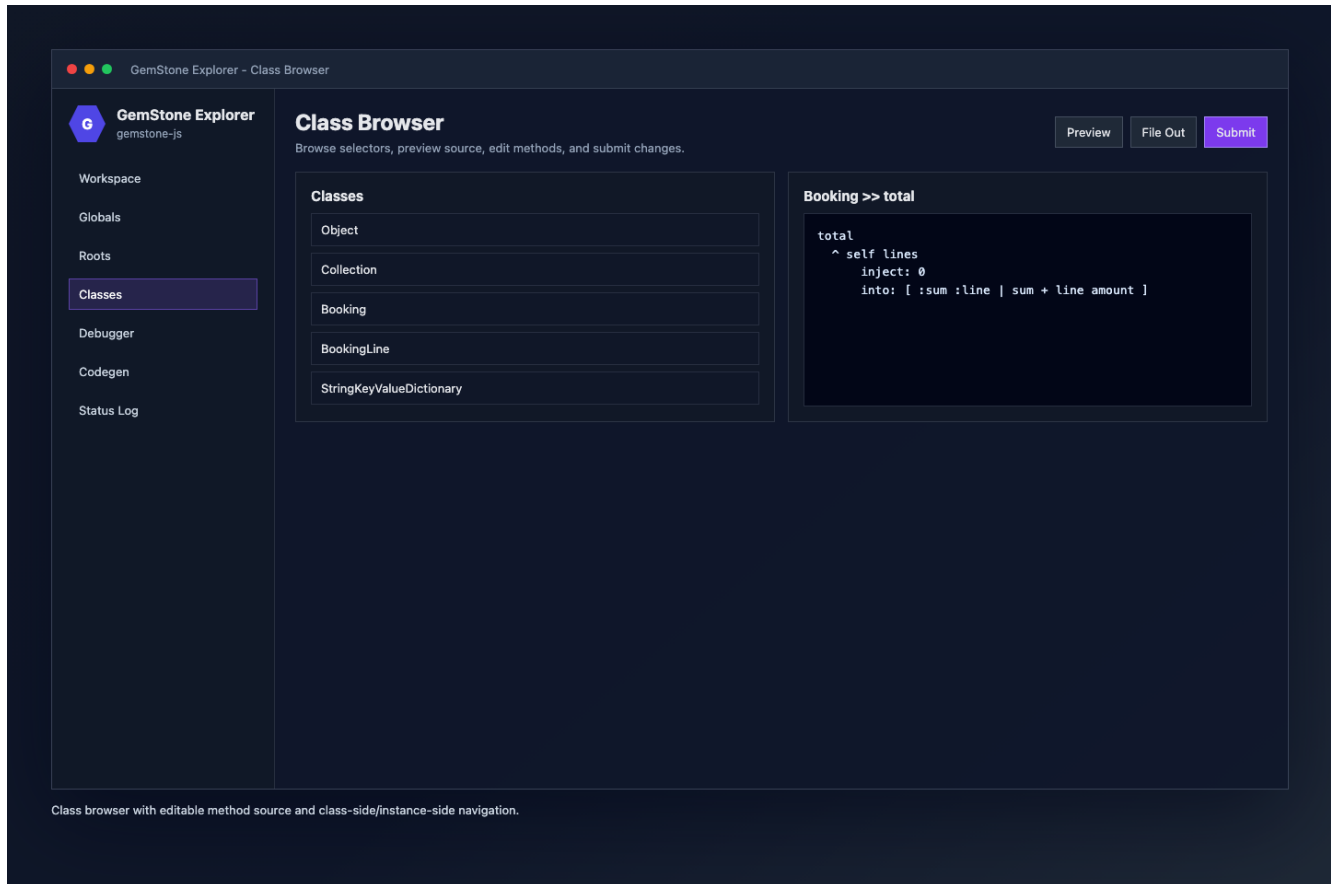
That is the core contract: keep the GemStone model visible, but make the JavaScript workflow predictable.

The Explorer

The Explorer is a local browser UI served by `examples/explorer.ts`. It gives the project a concrete development surface instead of only a library API.

It includes:

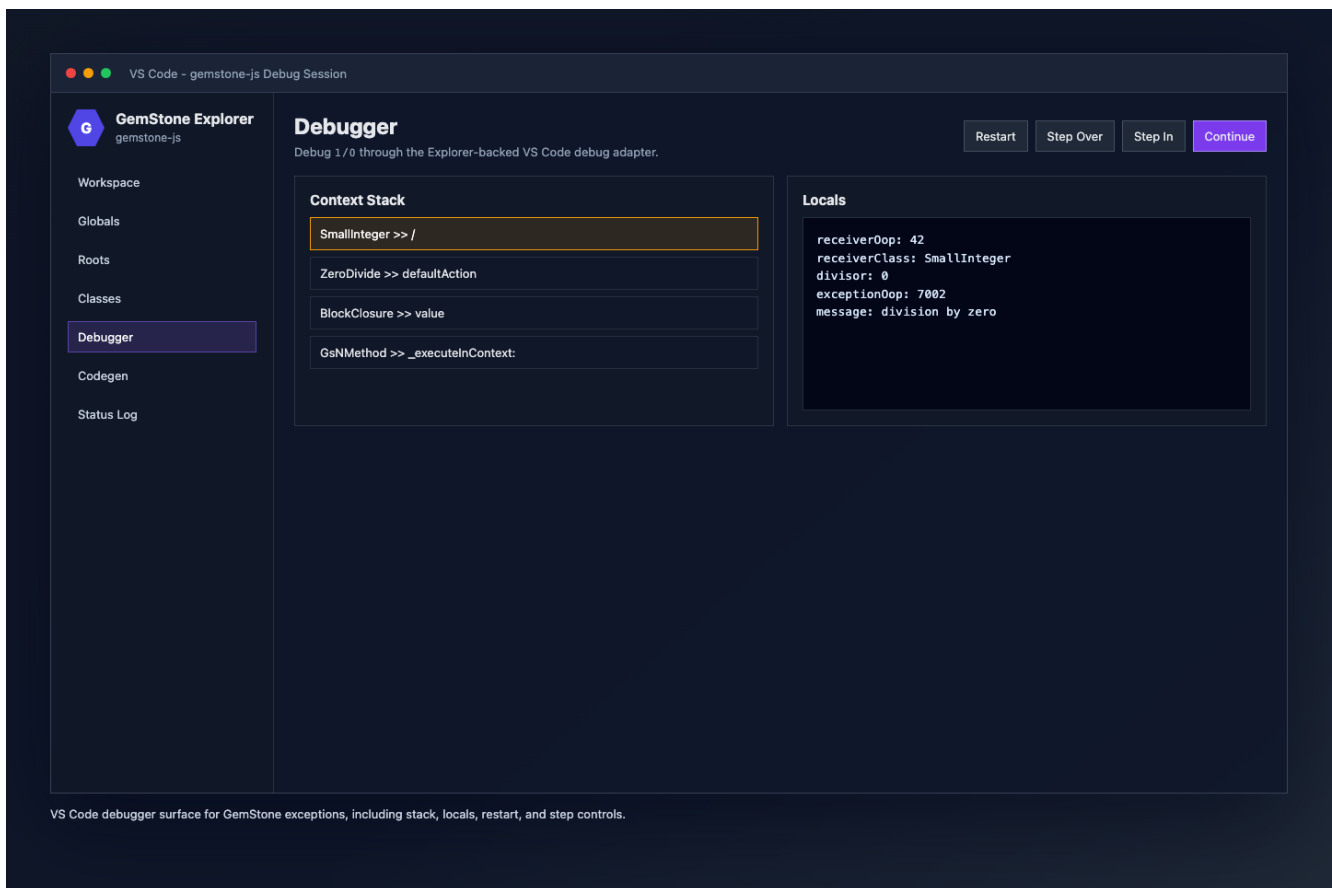
- connection status and Doctor diagnostics
- workspace evaluation with selectable return kinds
- roots and globals browsing
- object inspection by OOP
- class browsing and method source editing
- generated-wrapper preview
- debugger panels for exception contexts



The class browser is intentionally close to the Python explorer workflow: list classes, select a method, preview source, edit, and submit. Description and file-out views are available when needed, but the main surface stays focused on browsing and editing.

Debugging 1/0

Debugging is where the Explorer becomes more than a convenience UI. When evaluation raises, the workbench can open the debugger automatically. A simple 1/0 gives a live exception context with stack frames, locals, receiver details, and action buttons.



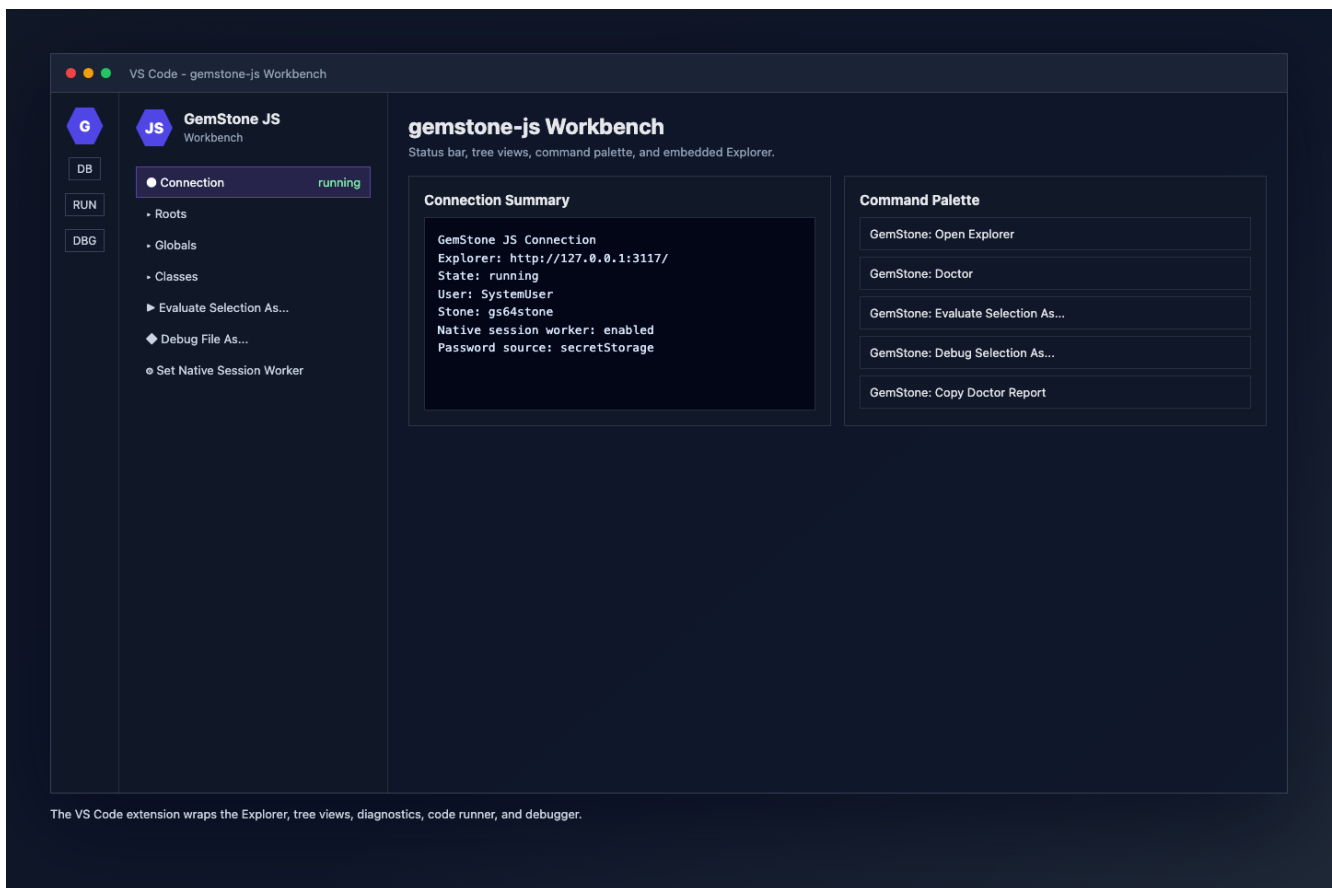
The debugger supports the operations developers expect first:

- continue
- restart
- step over
- step in
- step out
- selected-frame-aware actions
- source previews from GemStone context frames
- receiver, locals, context OOP, and exception OOP display

The debugger is deliberately thin. Session semantics remain owned by the Explorer and GemStone APIs; VS Code is a client surface.

The VS Code Wrapper

The VS Code extension started as a wrapper around the JavaScript Explorer rather than a completely separate UI. That was the right tradeoff.



The extension now provides:

- GemStone: Open Explorer
- GemStone: Doctor
- GemStone: Evaluate Selection
- GemStone: Evaluate Selection As...
- GemStone: Run File and Run File As...
- GemStone: Debug Selection and Debug Selection As...
- GemStone: Debug File and Debug File As...
- connection, roots, globals, and classes tree views
- object/class copy and inspect commands
- SecretStorage-backed password handling
- redacted connection and Doctor report copy commands
- quick settings for default return kind, Explorer open mode, and native session-worker mode

That gives the developer a familiar editor workflow without duplicating the Explorer's core features.

Why Return Kinds Matter

One of the most useful small details is return-kind selection.

When evaluating Smalltalk, developers often need different result shapes:

- **inspect** for a structured object inspection payload
- **value** for JavaScript value marshalling
- **oop** for a raw object handle

The workbench has a default return kind, but the **As...** commands let developers choose per action. That avoids a noisy settings loop when switching between quick checks, object inspection, and wrapper development.

Codegen and Reviewable APIs

The Rust article emphasizes checked-in generated wrappers, and the same idea is central in `gemstone-js`. A selector boundary should not live forever as hand-written stringly typed calls scattered through a service. Once a call becomes application API, put the selector spelling, argument names, return kind, and TypeScript imports in a manifest or decorated source file.

The manifest path is JSON and has a schema for editor validation:

```
{
  "$schema": "../schemas/codegen-manifest.schema.json",
  "imports": [
    {
      "from": "gemstone-js",
      "typeNames": ["Session", "TypedOop"]
    },
    {
      "from": "./booking.ts",
      "typeSpecifiers": [{ "name": "Booking" }]
    }
  ],
  "functions": [
    {
      "exportedName": "findBookingObject",
      "className": "Booking",
      "selector": "find:",
      "argNames": ["id"],
      "argTypes": ["string"],
      "sessionType": "Session",
      "returnType": "TypedOop<Booking>",
      "returnKind": "object"
    }
  ]
}
```

That renders a normal TypeScript function:

```
export async function findBookingObject(
  session: Session,
  id: string,
): Promise<TypedOop<Booking>> {
  return session.classRef("Booking").sendObject("find:", id);
}
```

The check commands keep generated output current:

```
npm run codegen -- examples/codegen.manifest.json
npm run codegen:check
npm run codegen:scan:check
```

The scanner path lets a TypeScript source file describe the same boundary with decorators:

```
import { GemStoneClass, GemStoneSelector, type Session, type TypedOop } from "gemstone-js";
import type { Booking } from "./booking.ts";

@GemStoneClass("Booking")
class BookingModel {
  @GemStoneSelector("find:")
  static findBookingObject(session: Session, id: string): Promise<TypedOop<Booking>> {
    throw new Error("Decorator source is scanned for codegen only.");
  }
}
```



```

    }
  }
}

```

The generated file is checked in. That is the important part: reviewers see exactly which selectors are exposed, which arguments are marshalled, and whether the result is a value, raw OOP, or retained typed handle.

Browser and Explorer Workflows

The Explorer is the UI expression of those same APIs. It can preview generated wrappers from a manifest, inspect object structure, browse classes and methods, edit method source, and open the debugger when evaluation raises. It is local tooling, not a separate application framework.

From a checkout, the browser UI starts as an installed example:

```
node --experimental-strip-types examples/explorer.ts
```

The example catalog prints the same command, so installed-package checks do not have to assume checkout-relative paths:

```
gemstone-js-examples --commands
gemstone-js-examples --show explorer
```

This is why the VS Code extension wraps the Explorer first. The workbench gets command-palette actions, tree views, SecretStorage, and debugger integration, while the tested Explorer still owns the browser UI and GemStone session operations.

Object Mapping Without an ORM

gemstone-js does support object mapping, but it does not try to make GemStone look like a generic JavaScript ORM.

The current model has three practical layers.

First, common JavaScript values are marshalled through `performWith()` and generated wrappers: strings, numbers, booleans, arrays, plain objects, dictionaries, and retained object handles cross the boundary without every caller manually allocating GemStone objects.

Second, live GemStone objects are represented as explicit handles:

```
import { Session, type TypedOop } from "gemstone-js";

interface Booking {
  id: string;
  status: string;
}

const session = await Session.connect();
const booking: TypedOop<Booking> = await session
  .classRef<Booking>("Booking")
  .sendObject("find:", "B-1001");

const status = await booking.send<string>("status");
await booking.send("status:", "confirmed");
await session.commit();
```

That `TypedOop<Booking>` is a retained remote object handle with a TypeScript witness. It is not a hydrated local `Booking` instance. Selector sends are still visible, async, and transaction-bound.

Third, generated wrappers turn those selector sends into application-facing functions:

```
export async function findBooking(
  session: Session,
  id: string,
```

```

): Promise<TypedOop<Booking>> {
  return session.classRef("Booking").sendObject("find:", id);
}

```

For a more transparent syntax, `mappedObject()` wraps the retained handle with async property-style methods:

```

import { mappedObject } from "gemstone-js";

const booking = mappedObject<Booking, {
  setStatus(status: string): Promise<unknown>;
}>(bookingObject, {
  setters: { setStatus: "status:" },
  snapshot: ["id", "status"],
});

const before = await booking.status();
await booking.setStatus("confirmed");
const payload = await booking.$snapshot();

```

That is intentionally not `await booking.status` or `booking.status = "confirmed"`. JavaScript property access is synchronous, but GemStone selector sends are remote and transaction-bound.

The newer `transparentObject()` layer goes further and matches gemstone-py's proxy shape more closely. Reads become awaitable properties, while callable accessors and explicit flush points keep JavaScript's async boundary visible:

```

import { transparentObject } from "gemstone-js";

const booking = transparentObject<Booking, {
  updateStatus(status: string, reason: string): Promise<string>;
}>(bookingObject, {
  selectors: { updateStatus: "status:reason:" },
  snapshot: ["id", "status"],
});

const before = await booking.status;
await booking.updateStatus("confirmed", "deposit received");
await booking.$assign({ status: "confirmed" });
await booking.$flush();
const payload = await booking.$snapshot();

```

Runtime assignment syntax is also supported by the proxy, but writes are queued because JavaScript setters cannot be async. Typed code can prefer `$assign()`, and UI/editor code can use assignment followed by `$flush()` when that reads better.

For explorer and migration workflows, `smalltalkBridge()` is closer to the dynamic bridge style in gemstone-py. Globals resolve lazily from properties, and JavaScript-friendly method names map underscores to Smalltalk keyword colons:

```

import { smalltalkBridge } from "gemstone-js";

const st = smalltalkBridge(session);

const objectClassName = await st.Object.name;
const array = await st.Array.new_.transparent<
  Record<string, unknown>,
  { size: PromiseLike<number> }
>(3);
await st.UserGlobals.at_put_("GemStoneJsBridgeDemo", 42);
const storedValue = await st.UserGlobals.at_("GemStoneJsBridgeDemo");

```

```
const arraySize = await array.size;
```

This is deliberately an opt-in tool and scripting layer. Stable application code can still use generated wrappers or `Session.classRef()`, while the Explorer and VS Code workbench can use the bridge for Smalltalk-like navigation and evaluation.

The project also includes a small compatibility facade for developers coming from **GemStone-Pharo-Bridge** and its MagLev branch examples. The goal is not to re-create Pharo in JavaScript, but to make the familiar session shape recognizable while using the same underlying `Session` and `PersistentRoot` APIs.

The classic example from `MAGLEV-BRANCH-USAGE.md` maps closely:

```
import { gbsSessionParameters } from "gemstone-js";

const session = await gbsSessionParameters()
  .name("Simple Session")
  .gemStoneName("gs64stone")
  .username("DataCurator")
  .password(process.env.GS_PASSWORD ?? "swordfish")
  .login();

try {
  await session.userGlobals.atPut("MyTestDict", {
    name: "Tariq",
    amount: 100,
    currency: "GBP",
  });

  await session.commit();

  const stored = await session.userGlobals.atDict("MyTestDict");
  console.log(stored ? await stored.toObject() : null);
  // { name: "Tariq", amount: 100, currency: "GBP" }
} finally {
  await session.disconnect();
}
```

That same root lookup works in a later session too. `atDict()` returns a `GsDict` handle, so the caller can either keep sending GemStone messages to the dictionary or call `toObject()` for a plain JavaScript snapshot.

The MagLev-oriented version keeps the same preference as the Pharo bridge guide: use `bridgeRoot` and explicit transaction behavior:

```
const session = await gbsSessionParameters()
  .name("MagLev Session")
  .gemStoneName("gs64stone")
  .username("DataCurator")
  .password(process.env.GS_PASSWORD ?? "swordfish")
  .netldiHostOrIp(process.env.GS_NETLDI_HOST ?? "localhost")
  .netldiNameOrPort(process.env.GS_NETLDI_NAME_OR_PORT ?? "50377")
  .login();

try {
  await session.bridgeRoot.atPut("MyTestDict", {
    name: "Tariq",
    amount: 100,
    currency: "GBP",
  });

  await session.commitTransactionOrSignalConflict();
}
```

```

const stored = await session.bridgeRoot.atDict("MyTestDict");
console.log(stored ? await stored.toObject() : null);
// { name: "Tariq", amount: 100, currency: "GBP" }
} finally {
  await session.disconnect();
}

```

The runnable packaged version is `examples/maglev-branch-usage.ts`. It preserves the original names, including `GbsSessionParameters`, `userGlobals`, `bridgeRoot`, `commit`, `commitTransactionOrSignalConflict`, and `disconnect`, while still allowing new code to move toward direct `Session`, `PersistentRoot`, generated wrapper, and transparent-object APIs.

For relationships and richer payloads, the same proxy can distinguish value, object, raw OOP, and dictionary readback explicitly:

```

import { mappedObject, type Oop, type TypedOop } from "gemstone-js";

const booking = mappedObject<Booking, {
  customer(): Promise<TypedOop<Customer>>;
  customerOop(): Promise<Oop>;
}>(bookingObject, {
  objectSelectors: { customer: "customer" },
  oopSelectors: { customerOop: "customer" },
  snapshot: {
    id: "id",
    status: "status",
    customer: { selector: "customer", kind: "oop" },
    details: { selector: "details", kind: "dict", maxEntries: 100 },
  },
});

const customer = await booking.customer();
const payload = await booking.$snapshot();

```

That gives the application a clear choice: keep a retained handle when it wants GemStone-side behavior, use a raw OOP when identity is enough, or produce a bounded snapshot when sending data to a UI or API.

The same rule applies when a root or global is the entry point. Read a retained handle from the root, wrap it for domain methods, commit or abort through the same session, and release the handle when the request is done:

```

import { PersistentRoot, mappedObject } from "gemstone-js";

const root = PersistentRoot.userGlobals(session);
const object = await root.requireObject<Booking>("LastBooking");
const booking = mappedObject<Booking, {
  setStatus(status: string): Promise<unknown>;
}>(object, {
  setters: { setStatus: "status:" },
  snapshot: ["id", "status"],
});

try {
  await booking.setStatus("confirmed");
  await booking.$session.commit();
} finally {
  await booking.$release();
}

```

The practical migration path is to start with `TypedOop<T>.send()`, move repeated selector sets into

mappedObject() options, then generate BookingRef classes when the mapping becomes shared API. For payload-style mapping, class instances can be converted explicitly into GemStone dictionaries:

```
import {
  Session,
  objectToDictionaryArgument,
  scalarValueConverterRegistry,
} from "gemstone-js";

class BookingDraft {
  constructor(
    public id: string,
    public status: string,
    public requestedAt: Date,
  ) {}
}

const session = await Session.connect({
  valueConverters: scalarValueConverterRegistry(),
});

const draft = new BookingDraft("B-1001", "held", new Date());
const payload = objectToDictionaryArgument(draft);
const booking = await session
  .classRef<Booking>("Booking")
  .sendObject("createFromDictionary:", payload);
```

Dictionaries are the other common mapping target. A StringKeyValueDictionary can hold marshalled values, nested dictionaries, and object handles while keeping the access mode explicit:

```
import { PersistentRoot } from "gemstone-js";

const dict = await session.dictionary({
  status: "held",
  tags: ["vip", "late-arrival"],
  limits: { guests: 2, bags: 3 },
});

await dict.setObject("booking", booking);
await dict.setAllValue({
  owner: "front-desk",
  priority: 3,
});

const status = await dict.requireValue("status");
const sameBooking = await dict.requireObject<Booking>("booking");
const limits = await dict.requireDict("limits");
const entries = await dict.items({ maxEntries: 50 });
const rawEntries = await dict.itemsOop({ maxEntries: 50 });

const root = PersistentRoot.userGlobals(session);
const index = await root.setDict("BookingIndex", {
  kind: "booking-index",
  active: true,
});

await index.setObject("B-1001", sameBooking);
```

```
await root.setObject("LastBooking", sameBooking);
```

The API names do real work here. `Value` methods marshal back to JavaScript values, `Object` methods return retained `TypedOop<T>` handles, `Oop` methods preserve raw identity, and `Dict` methods wrap nested dictionaries.

This is the same bias as the rest of the project: preserve `GemStone` identity and transaction semantics, but make the JavaScript boundary typed, repeatable, and easy to review.

With `mappedObject()` in place, the next mapping step should borrow the useful part of the Pharo bridge connector model without copying its transparent synchronization semantics. The shape is straightforward:

- the committed `schemas/object-mapping-manifest.schema.json` contract and `examples/object-mapping.manifest.js` example for class name, TypeScript type, selectors, setters, repository methods, and snapshot fields
- generated `BookingRef`-style classes wrapping `TypedOop<T>` or delegating to `mappedObject()`
- repository helpers returning typed refs
- bounded `snapshot()` and dictionary helpers for UI/API payloads
- Explorer and VS Code mapping views over committed manifests and generated files

That would let application code say:

```
const bookings = new BookingRepository(session);
const booking = await bookings.find("B-1001");

await booking.setStatus("confirmed");
const payload = await booking.snapshot();
```

The important constraint is that `status()` and `setStatus()` stay async remote calls. The mapping layer should make selector use nicer, not pretend the `GemStone` object is a local JavaScript object.

Native Worker Mode

The native backend is the main production hardening area. The raw GCI binding remains useful for low-level troubleshooting, but production trials should use the native session worker when possible.

The worker model queues calls for a session through a dedicated native worker boundary. This is a better fit for blocking native calls and `GemStone` session expectations, and it keeps JavaScript call sites async.

The workbench exposes this as a quick setting:

GemStone: Set Native Session Worker

Changing it stops the managed Explorer so the next launch uses the new mode.

Release Confidence

The project now treats release proof as part of the library, not an afterthought.

The local verification path checks:

- TypeScript typecheck
- codegen manifest output
- decorator scanner output
- examples catalog
- comparison reports
- public API surface
- runtime API contract
- unit tests
- live-test guard coverage
- checksum and provenance helpers
- package artifact review
- installed API contract

The VS Code workbench has its own verify path that checks syntax, offline extension activation, command registration, VSIX packaging, artifact contents, and optional extension-host smoke tests.

What This Enables

The important shift is that GemStone/S work can now sit inside a modern JavaScript delivery loop:

- write TypeScript services
- generate typed wrappers for GemStone selectors
- run local tests without a live Stone
- opt into live Stone regression checks
- inspect production-shaped objects through the Explorer
- debug exceptions from VS Code
- publish npm packages with artifact/provenance checks

That does not make GemStone less GemStone. It makes the surrounding workflow easier to ship and trust.

What Comes Next

The next valuable work is less about adding more small commands and more about hardening:

- deeper live regression coverage
- stronger native worker stress tests
- installed-package smoke checks across target platforms
- Marketplace release polish for the VS Code extension
- connector-inspired object mapping with generated `*Ref` classes and snapshot helpers
- more visual Explorer workflows once the core debugger/class browser semantics settle

The workbench is already useful as a wrapper around the Explorer. That is the right MVP. From there, the extension can grow into a richer GemStone development environment without losing the tested browser UI beneath it.

Links

- `gemstone-js`: <https://github.com/unicompute/gemstone-js>
- VS Code workbench package: `vscode-gemstone-js-workbench`
- Generated PDF bundle: `docs/pdf`